RoadSOS

Hackathon Submission Package

National Road Safety Hackathon 2026

CoERS — Centre of Excellence in Road Safety
IIT Madras

https://coers.iitm.ac.in/events/Hackathon/2026/rule_book/

Submission deadline: 31 May 2026 **Repository:** https://github.com/Arthrevs/RoadSOS

Live app: https://roadsos-frontend.vercel.app

Section	Contents
1. Project overview	Problem, judging-criteria map
2. Assumptions	27 explicit design constraints
3. Software packages	Backend pip + frontend npm + external APIs
4. Architecture	Request flow, offline strategy, crash detection
5. Key source code	Plus Code encoder + AI triage (full repo on GitHub)

Contents

1 Project overview 2
1.1 Judging-criteria map . . . 2

2 Assumptions 2
2.1 Network and connectivity . . . 2
2.2 Device and browser . . . 2
2.3 Geolocation . . . 3
2.4 Geography and data coverage . . . 3
2.5 User behaviour and consent . . . 3
2.6 Communications channels . . . 3
2.7 AI triage . . . 3
2.8 Hosting . . . 4
2.9 Security and privacy . . . 4

3 Software packages 4
3.1 Backend — Python runtime (`backend/requirements.txt`) . . . 4
3.2 Backend — Python development tools (`backend/requirements-dev.txt`) . . . 4
3.3 Frontend — npm runtime dependencies . . . 4
3.4 Frontend — npm development dependencies . . . 5
3.5 External services (consumed via REST — no SDK) . . . 5

4 Architecture summary 5
4.1 Request flow — `GET /search` . . . 5
4.2 Four-tier offline strategy . . . 6
4.3 Crash detection . . . 6
4.4 SOS dispatch . . . 6

5 Key source code 6
5.1 Offline Plus Code encoder . . . 7
5.1.1 `frontend/src/utils/plusCodes.js` [JavaScript, 3,825 chars] . . . 7
5.2 AI triage logic . . . 8
5.2.1 `backend/services/ai_triage.py` [Python, 7,582 chars] . . . 8
5.3 Source index . . . 12

1 Project overview

RoadSOS — emergency response and prioritised contact discovery for road accidents.

India records 1.5 lakh road-accident deaths per year. The first 60 minutes after a severe injury (the *golden hour*) determines survival. At a real crash scene a bystander currently needs 2–3 minutes just to find the right phone number via Google Maps. RoadSOS reduces that to under 10 seconds and works fully offline.

1.1 Judging-criteria map

Criterion (from rulebook)	How RoadSOS satisfies it
Number of emergency contacts found	OSM Overpass (9 categories, parallel) + Google Places (parallel, not sequential). Auto-expanding radius 5 km $\to$ 10 km $\to$ 20 km.
Reliability	Every external call wrapped in `_safe_*` helpers. 3-mirror Overpass with exponential back-off. API always returns HTTP 200 with a valid shape.
Offline functionality	Four-tier fallback: live backend $\to$ 24 h localStorage cache $\to$ bundled JSON (818 entries, 200 countries) $\to$ hardcoded mock. Country emergency numbers always offline.
International coverage	200 countries pre-loaded. ISO-3166 code from Nominatim. Emergency numbers switch automatically at borders.
Six mandatory service categories	`hospital`, `police`, `ambulance`, `towing`, `tyre`, `showroom` — all mapped to OSM tags and Google keyword queries.

2 Assumptions

The following assumptions underpin the design. The code degrades gracefully whenever any single assumption is violated at runtime.

2.1 Network and connectivity

- The user's device has at least intermittent connectivity at first install so the service worker can precache the shell and bundled facilities. After install the app is fully usable offline.
- When online, OpenStreetMap Overpass and Google Places are reachable; when not, the four-tier fallback (live backend localStorage cache bundled JSON hardcoded mock) ensures contacts are always returned.
- Render's free-tier backend spins down after 15 min of inactivity; the first request takes up to 55 s. The frontend fires a warm-up ping and shows a status banner explaining this.

2.2 Device and browser

- Modern evergreen browsers (Chrome, Edge, Firefox, Safari 14). Service Workers, Geolocation API, and Web Speech API are required for full functionality.
- Battery Status API is supported on Chromium-based browsers but not iOS Safari or desktop Firefox. The battery row in the dispatch screen is hidden when the API is unavailable.

- Accelerometer access (crash detection) requires the DeviceMotion API. iOS Safari requires an explicit permission prompt; the app degrades gracefully when denied.

2.3 Geolocation

- GPS accuracy is within 50 m in open sky, 200 m in urban canyons. The app commits a coarse fix in under 2 s and refines with watchPosition.
- iCloud Private Relay and similar proxies can return IP-geolocated positions hundreds of kilometres from the device. A three-layer guard (org-name check, >500 km haversine drift, Apple datacentre bbox) rejects these and re-prompts for GPS.
- Locations are cached at ~110 m grid resolution (3 decimal places) coarse enough for high cache hit-rate, fine enough for emergency-contact relevance.

2.4 Geography and data coverage

- OSM data quality is best in India, Europe, and dense urban areas. In sparse rural regions the Overpass radius expands 8 km 25 km and falls back on bundled facilities (938 entries across 200 countries).
- Country emergency numbers are pre-bundled for all 200 countries and always render without a network call.
- ISO-3166 country code is resolved from Nominatim reverse-geocode; if Nominatim fails the app falls back to coarse bounding-box matching against the bundled country dataset.

2.5 User behaviour and consent

- Users grant geolocation permission on first launch; otherwise the app shows a Set-Location-Manually affordance.
- Medical ID data stored in localStorage is included in outgoing SOS payloads only after the user sets it up. It is never uploaded to RoadSOS servers.
- On auto-detected crash (sustained 40 km/h collapsing to 5 km/h within ~2.5 s, optionally confirmed by a 3.5 G accelerometer spike), a cancellation window lets the user abort false positives.

2.6 Communications channels

- Country code from Nominatim determines the primary dispatch channel: WhatsApp-dominant countries (India, Brazil, Indonesia, Nigeria, etc.) use wa.me links; elsewhere native SMS deep-links are used.
- If the WhatsApp app is not installed, the wa.me link opens WhatsApp Web. The handler deferred-falls-back to SMS after 1.2 s if the WA window failed to open.
- The window.open call for WhatsApp fires synchronously inside the click handler any await before it strips the user-gesture flag and causes browsers to silently block the popup.

2.7 AI triage

- Google Gemini 2.0 Flash is used for situational triage via direct REST (no SDK). Free-tier limits (60 RPM / 1500 RPD) are sufficient for a demo.
- If Gemini is unreachable or rate-limited the app falls back to deterministic rule-based prioritisation using the same two yes/no questions.

2.8 Hosting

- Frontend is on Vercel (auto-deploy from main). Backend is on Render (auto-deploy from main). Both are free tiers; production would migrate the backend to a paid host to eliminate cold starts.
- The frontend can be served from any static host; the backend is a vanilla FastAPI app deployable on any Python 3.11+ host with uvicorn.

2.9 Security and privacy

- Medical ID, language preference, and cached searches live in localStorage only. The backend is stateless and never persists user data.
- Rate limits (/search 30 rpm/IP, /triage 20 rpm/IP) are enforced by token-bucket middleware aware of Render's X-Forwarded-For header.
- Google Places API keys are server-side only; they are rotated per request; if a key returns 403 the rotation picks the next key. The frontend never sees them.

3 Software packages

3.1 Backend — Python runtime (`backend/requirements.txt`)

```
fastapi==0.115.0
uvicorn[standard]==0.30.6
httpx==0.27.2
python-dotenv==1.0.1
aiosqlite==0.20.0
pydantic==2.9.2
phonenumbers==8.13.46
unidecode==1.3.0
```

3.2 Backend — Python development tools (`backend/requirements-dev.txt`)

```
-r requirements.txt
pytest==8.3.3
pytest-asyncio==0.24.0
# TestClient requires httpx (already in requirements) [U+2014] no extra dep needed
ruff==0.11.13
```

3.3 Frontend — npm runtime dependencies

Package	Version
`i18next`	^26.2.0
`leaflet`	^1.9.4
`lucide-react`	^1.16.0
`react`	^18.3.1
`react-dom`	^18.3.1
`react-i18next`	^17.0.8
`react-joyride`	^3.1.0
`react-leaflet`	^4.2.1
`react-router-dom`	^7.15.0
`workbox-window`	^7.4.1

3.4 Frontend — npm development dependencies

Package	Version
`@types/react`	^18.3.3
`@types/react-dom`	^18.3.0
`@vitalets/google-translate-api`	^9.2.1
`@vitejs/plugin-react`	^4.7.0
`jsdom`	^29.1.1
`react-refresh`	^0.18.0
`vite`	^7.3.3
`vite-plugin-pwa`	^1.3.0
`vitest`	^2.1.4
`workbox-expiration`	^7.4.1
`workbox-precaching`	^7.4.1
`workbox-routing`	^7.4.1
`workbox-strategies`	^7.4.1

3.5 External services (consumed via REST — no SDK)

Service / API	Role in RoadSOS
OpenStreetMap Overpass API	Public OSM data query. Three mirrors with exponential-backoff retry.
OpenStreetMap Nominatim	Reverse geocoding (lat/lon → country code, landmark).
Google Places API (Nearby Search + Place Details)	Unconditional parallel fallback (if configured).
Google Gemini 2.0 Flash	AI triage. Free tier: 60 RPM / 1 500 RPD.
CartoDB Dark Matter tiles	Map tile provider (no API key required).
Web platform APIs	Geolocation, DeviceMotion, Battery Status, Web Speech, Service Worker.

4 Architecture summary

The full 35-page technical reference is at `docs/TECHNICAL.pdf` in the repository. The summary below captures the request flow and the four-tier offline strategy.

4.1 Request flow — `GET /search`

1. Frontend issues `GET /search?lat=...&lon=...&radius=5000`.
2. Phase 1: parallel fan-out — Nominatim reverse geocode + Overpass query (9 OSM categories, 3 mirrors, 12 s timeout each).
3. Phase 2: parallel Google Places Nearby Search if a key is configured (does **not** wait for OSM; `rankby=distance` for nearest-first).
4. Phase 3: merge + proximity dedupe (50 m radius), then phone enrichment via up to 3 Place Details calls within a 5 s budget.

5. Response: JSON with categorised contacts, country emergency numbers, landmark, ISO country code.
6. Frontend: render on Leaflet map (CartoDB Dark Matter tiles), populate Quick-Contacts dock, persist to 24 h localStorage cache.

4.2 Four-tier offline strategy

1. **Tier 1** — Live backend `/search` call (online path).
2. **Tier 2** — Service Worker + localStorage cache, 24 h TTL, keyed at ~1.1 km grid.
3. **Tier 3** — Bundled JSON shipped with the PWA: 818 entries across 200 countries.
4. **Tier 4** — Hardcoded mock so the UI never shows an empty state.

4.3 Crash detection

- **Signal 1 (primary)** — GPS velocity collapse: sustained ≥40 km/h → ≤5 km/h within 2.5 s. Tuned to reject ordinary braking and stop-and-go traffic.
- **Signal 2 (optional)** — Accelerometer spike: peak magnitude ≥3.5 G.
- **Confirmation**: the accelerometer spike, when motion permission is granted, must align within a 4 s window; a sustained-speed sudden stop alone still triggers.
- **Cooldown**: 12 s after confirmed event.
- **User override**: 10 s PIN-cancel window before auto-dispatch.

4.4 SOS dispatch

- WhatsApp-dominant countries (India, Brazil, Indonesia, Nigeria, . . .): `wa.me` deep-link opened **synchronously** in the user-gesture frame.
- SMS-dominant countries: native `sms:` deep-link.
- Fallback: if the WA popup is closed within 1.2 s the handler fires an SMS fallback URL via `window.location.href`.
- After the synchronous dispatch: camera + torch + scene-photo capture, live tracking session creation, and DispatchScreen overlay all run asynchronously without blocking the open tab.

5 Key source code

The two algorithmic centrepieces are included verbatim below: the fully-offline Plus Code (Open Location Code) encoder and the AI triage logic. The complete 146-file source tree and full Git history are on GitHub: https://github.com/Arthrevs/RoadSOS.

5.1 Offline Plus Code encoder

5.1.1 frontend/src/utils/plusCodes.js [JavaScript, 3,825 chars]

```
/**
 * Open Location Code (Plus Codes) [U+2014] encode GPS coordinates to a short,
 * speakable, dispatcher-friendly string. Algorithm by Google. Spec:
 *   https://github.com/google/open-location-code/blob/main/docs/specification.md
 *
 * Why we ship this:
 * - Indian emergency dispatchers (esp. 112 ERSS) accept Plus Codes.
 * - "7M5237MC+37" is far easier to communicate by voice in a panicked
 *   moment than "thirteen point zero eight two seven, eighty point two
 *   seven zero seven".
 * - **Fully offline** [U+2014] pure deterministic algorithm. No network, no
 *   API key, no library.
 *
 * Implementation note:
 * - Uses the reference INTEGER algorithm (multiply to a fixed precision,
 *   then divide). Float-accumulation approaches drift on the final grid
 *   digits, so we deliberately avoid them. Verified character-for-character
 *   against Google's canonical encoder across 1000+ coordinates.
 *
 * Precision: length 10 (default) [U+2248] 14 m [U+00D7] 14 m square [U+2014] fine for a
    ↪ crash site.
 *
 * Exports `encodePlusCode(lat, lon)` [U+2192] 11-char code, e.g. "7M5237MC+37".
 */

const ALPHABET = '23456789CFGHJMPQRVWX'; // 20 chars, skips 0/I/lookalikes
const SEPARATOR = '+';
const SEPARATOR_POSITION = 8;
const ENCODING_BASE = 20;
const LATITUDE_MAX = 90;
const LONGITUDE_MAX = 180;
const PAIR_CODE_LENGTH = 10;
const GRID_CODE_LENGTH = 5;
const GRID_COLUMNS = 4;
const GRID_ROWS = 5;

// Precision multipliers (integer algorithm).
const PAIR_PRECISION = ENCODING_BASE ** 3;
const FINAL_LAT_PRECISION = PAIR_PRECISION * GRID_ROWS ** GRID_CODE_LENGTH;
const FINAL_LON_PRECISION = PAIR_PRECISION * GRID_COLUMNS ** GRID_CODE_LENGTH;

function clipLatitude(lat) {
  return Math.max(-90, Math.min(90, lat));
}

function normalizeLongitude(lon) {
  let l = lon;
  while (l < -180) l += 360;
  while (l >= 180) l -= 360;
  return l;
}

/**
 * Encode (lat, lon) to a 10-digit Plus Code with the "+" separator.
 *
 * @param {number} lat [U+2014] degrees, -90 to 90 (clamped)
 * @param {number} lon [U+2014] degrees, wrapped to [-180, 180)
 * @returns {string} e.g. "7M5237MC+37", or "" for invalid input
 */
export function encodePlusCode(lat, lon) {
  if (typeof lat !== 'number' || typeof lon !== 'number'
      || !isFinite(lat) || !isFinite(lon)) {
    return '';
  }

  let latitude = clipLatitude(lat);
  const longitude = normalizeLongitude(lon);

  // Latitude 90 would overflow the top cell [U+2014] pull it inward by one cell.
  if (latitude === 90) {
    latitude = latitude - (ENCODING_BASE ** -3 / GRID_ROWS ** GRID_CODE_LENGTH);
  }
```

```

  // Convert to positive integers at the final precision. Integer-only math
  // from here avoids floating-point drift on the trailing grid digits.
  let latVal = Math.floor(
    Math.round((latitude + LATITUDE_MAX) * FINAL_LAT_PRECISION * 1e6) / 1e6,
  );
  let lonVal = Math.floor(
    Math.round((longitude + LONGITUDE_MAX) * FINAL_LON_PRECISION * 1e6) / 1e6,
  );

  // A length-10 code uses only the pair section, so drop the grid digits.
  latVal = Math.floor(latVal / GRID_ROWS ** GRID_CODE_LENGTH);
  lonVal = Math.floor(lonVal / GRID_COLUMNS ** GRID_CODE_LENGTH);

  // Build the 5 lat/lon pairs from least- to most-significant.
  let code = '';
  for (let i = 0; i < PAIR_CODE_LENGTH / 2; i++) {
    code = ALPHABET.charAt(lonVal % ENCODING_BASE) + code;
    code = ALPHABET.charAt(latVal % ENCODING_BASE) + code;
    latVal = Math.floor(latVal / ENCODING_BASE);
    lonVal = Math.floor(lonVal / ENCODING_BASE);
  }

  // Insert the "+" separator after position 8.
  return code.slice(0, SEPARATOR_POSITION) + SEPARATOR + code.slice(SEPARATOR_POSITION);
}

/**
 * Build a Google Maps shareable URL for given coordinates.
 * Used by SOS-by-SMS so the recipient can tap-to-open the location.
 */
export function gMapsLink(lat, lon) {
  return `https://maps.google.com/?q=${lat.toFixed(6)},${lon.toFixed(6)}`;
}
```

5.2 AI triage logic

5.2.1 backend/services/ai_triage.py [Python, 7,582 chars]

```python
"""AI-driven contact prioritisation.

Uses Google's Gemini 2.5 Flash for situation-aware reordering. Always falls
back to a deterministic rule-based ordering if the API is unreachable, errors,
or returns malformed output. The fallback produces the same response shape so
the frontend never sees a difference.

Why fallback matters: emergency software cannot have its core feature dependent
on a 3rd-party API.

Why Gemini Flash: free tier (15 RPM, 1500 RPD on 2.5-flash, 15 RPM on 1.5-flash),
low latency, JSON-mode supported, no billing required to start.
"""

from __future__ import annotations

import json
import logging
import os
import re

import httpx

from services.gemini_quota import gemini_quota
from services.gemini_utils import extract_gemini_text

logger = logging.getLogger(__name__)

# Gemini 2.5 Flash [U+2014] current free-tier default with the highest free quota.
MODEL = "gemini-2.5-flash"
MAX_TOKENS = 2048
TIMEOUT_S = 15.0

# REST endpoint (no SDK dependency [U+2014] keeps requirements lean and matches our
# existing httpx-everywhere style).
```

~~~~
GEMINI_URL = (
    "https://generativelanguage.googleapis.com/v1beta/models/{model}:generateContent?key={key}"
)

SYSTEM_PROMPT = """You are RoadSOS Triage [U+2014] an emergency dispatcher AI for road accidents.

Your only job: take a crash situation and a list of nearby emergency services,
return a JSON object that reorders the services by priority for that specific
situation. Uses Gemini 2.5 Flash to reorder the contacts for the specific situation.

OUTPUT REQUIREMENTS (strict):
- Return ONLY a JSON object. No prose. No markdown fences. No commentary.
- The object MUST have exactly two keys: "contacts" and "reason".
- "contacts" MUST be the same array of objects you were given, with all fields
  preserved verbatim, just reordered. Never invent contacts. Never drop them.
- "reason" MUST be a single sentence, maximum 18 words, plain English,
  explaining why the top contact was chosen for this situation.

PRIORITY RULES (in strict order):
1. Injured + vehicle blocking road  [U+2192] ambulance [U+2192] hospital [U+2192] police [U+2192] towing [U+2192] repair
2. Injured + no road block          [U+2192] ambulance [U+2192] hospital [U+2192] police [U+2192] repair
3. Not injured + blocking road      [U+2192] police [U+2192] towing [U+2192] repair [U+2192] tyre
4. Not injured + no block           [U+2192] repair [U+2192] tyre [U+2192] police [U+2192] towing

Within a priority tier, the closer service wins."""


_FENCE_RE = re.compile(r"^```(?:json)?\s*|\s*```$", re.MULTILINE)


def rule_based_triage(injured: bool, blocking: bool, contacts: list[dict]) -> dict:
    """Deterministic ordering used as fallback and as a baseline for tests."""
    if injured and blocking:
        order = ["ambulance", "hospital", "police", "towing", "repair", "tyre", "showroom"]
        reason = "Trauma care plus blocked road [U+00B7] ambulance, hospital, police listed first"
    elif injured:
        order = ["ambulance", "hospital", "police", "repair", "towing", "tyre", "showroom"]
        reason = "Trauma care prioritised [U+00B7] ambulance and hospital listed first"
    elif blocking:
        order = ["police", "towing", "repair", "tyre", "hospital", "ambulance", "showroom"]
        reason = "Vehicle blocking traffic [U+00B7] police and towing listed first"
    else:
        order = ["repair", "tyre", "showroom", "police", "towing", "hospital", "ambulance"]
        reason = "No injuries reported [U+00B7] roadside repair services listed first"

    def priority(c: dict) -> tuple:
        cat = c.get("category", "")
        idx = order.index(cat) if cat in order else 99
        return (idx, c.get("distance", 9999))

    return {
        "contacts": sorted(contacts, key=priority),
        "reason": reason,
    }


def _validate_ai_result(result: object, original_count: int) -> dict | None:
    """Return the result if it matches the contract, else None."""
    if not isinstance(result, dict):
        return None
    if "contacts" not in result or "reason" not in result:
        return None
    if not isinstance(result["contacts"], list):
        return None
    if not isinstance(result["reason"], str) or not result["reason"].strip():
        return None
    if len(result["contacts"]) != original_count:
        return None
    return result
~~~~

```python

async def prioritize_contacts(injured: bool, blocking: bool, contacts: list[dict]) -> dict:
    if not contacts:
        return {"contacts": [], "reason": "No nearby services found"}

    api_key = os.getenv("GEMINI_API_KEY", "")
    if not api_key:
        logger.info("GEMINI_API_KEY not set [U+2014] using rule-based triage")
        return rule_based_triage(injured, blocking, contacts)

    if not gemini_quota.can_call():
        logger.info("Gemini quota guard triggered [U+2014] using rule-based triage")
        return rule_based_triage(injured, blocking, contacts)

    try:
        situation = []
        if injured:
            situation.append("PEOPLE ARE INJURED and need medical attention")
        if blocking:
            situation.append("THE VEHICLE IS BLOCKING THE ROAD")
        if not situation:
            situation.append("no injuries, vehicle may need roadside assistance")
        situation_text = "; ".join(situation)

        summary = [
            {
                "name": c.get("name", "?"),
                "category": c.get("category", "?"),
                "distance_km": c.get("distance", "?"),
            }
            for c in contacts
        ]

        user_message = (
            f"SITUATION: {situation_text}.\n\n"
            f"Services found nearby (currently sorted by distance only):\n"
            f"{json.dumps(summary, indent=2)}\n\n"
            f"Full contact data to reorder (return ALL of these, "
            f"just in the priority order you decide):\n"
            f"{json.dumps(contacts, indent=2)}"
        )

        # Gemini puts the system prompt in `systemInstruction`, not in messages.
        # JSON mode is opt-in via responseMimeType.
        payload = {
            "systemInstruction": {"parts": [{"text": SYSTEM_PROMPT}]},
            "contents": [
                {"role": "user", "parts": [{"text": user_message}]},
            ],
            "generationConfig": {
                "temperature": 0.2,
                "maxOutputTokens": MAX_TOKENS,
                "responseMimeType": "application/json",
                "thinkingConfig": {"thinkingBudget": 0},
            },
        }

        url = GEMINI_URL.format(model=MODEL, key=api_key)
        async with httpx.AsyncClient(timeout=TIMEOUT_S) as client:
            r = await client.post(url, json=payload)
            r.raise_for_status()
            response_json = r.json()
        await gemini_quota.record_call()

        raw = extract_gemini_text(response_json)
        if not raw:
            logger.warning("AI returned empty response [U+00B7] using rule-based fallback")
            return rule_based_triage(injured, blocking, contacts)

        cleaned = _FENCE_RE.sub("", raw).strip()

        try:
            parsed = json.loads(cleaned)
        except json.JSONDecodeError:
            logger.warning("AI returned non-JSON [U+00B7] using rule-based fallback")
            return rule_based_triage(injured, blocking, contacts)

```

```
            validated = _validate_ai_result(parsed, len(contacts))
            if validated is None:
                logger.warning("AI returned malformed response [U+00B7] using rule-based fallback")
                return rule_based_triage(injured, blocking, contacts)

            return validated

    except Exception as exc:
        logger.warning(f"AI triage call failed ({type(exc).__name__}) [U+00B7] using rule-based fallback")
        return rule_based_triage(injured, blocking, contacts)
```

5.3 Source index

Files included verbatim: **2**.

Document version. Generated from `main`. Re-run `python scripts/build_submission_tex.py` after any code change to regenerate.